Programming Techniques 2026–2027

Lecture 7: Formatted and File I/O, and Makefiles

Hannu Parviainen

Universidad de la Laguna

October 19, 2026

Recap: basic I/O

- In `read(unit, fmt)` and `write(unit, fmt)`, the unit comes first and the format second.
- A `*` unit is the terminal. A `*` format is list-directed: the compiler chooses the layout.
- `iostat=ios` is 0 on success, negative at the end of the input, and positive on an error. `iomsg=msg` gives the message.
- An internal read reads from a string: `read(line, *) x`.
- Redirection: `./a.out < in.txt > out.txt`.

Statement	Unit	Format
`print *, x`	std. output	list-directed
`write(*, *) x`	std. output	list-directed
`read *, x`	std. input	list-directed
`read(*, *) x`	std. input	list-directed
`read(line, *) x`	a string	list-directed
`write(u, '(f8.3)') x`	a file	explicit

Today: explicit formats, files, command-line arguments, and building programs with `make`.

Formatted Input and Output

Format specifications

- A format is a string in parentheses that replaces the * format.
- It has one *edit descriptor* for each item. Quoted text inside it is written as it is.
- A format can be stored in a named constant and reused.
- Formatted output does not start with a space.
- Old code uses labelled `format` statements: `write(*, 100) x` with `100 format(f8.3)`.

```
program formats
  implicit none
  character(len=*), parameter :: fmt = '(a, f8.3, a)'
  real :: period = 365.25636

  print *, 'Period:', period, 'days'
  print '(a, f8.3, a)', 'Period:', period, ' days'
  write(*, fmt) 'Period:', period, ' days'
  write(*, '("Period: ", f0.1, " days")') period
end program formats
```

```
$ gfortran formats.f90 && ./a.out
 Period:   365.256348     days
Period: 365.256 days
Period: 365.256 days
Period: 365.3 days
```

Edit descriptors

`Iw`	integer, field width `w`
`Iw.m`	integer, at least `m` digits
`Fw.d`	real, `d` decimals
`ESw.d`	real, scientific notation
`Ew.d`	real, exponential notation
`Aw`	string (`A`: its own length)
`Lw`	logical, `T` or `F`
`G0`	any type, minimal width
`nX`	skip `n` positions
`/`	start a new line
`"text"`	the text itself

- The width `w` includes the sign and the decimal point.
- Values are right-aligned in their field.

```
program descriptors
  implicit none
  integer :: n = 42
  real :: d = 456.789, m = 1.989e30
  print '(i5, "|", i5.4, "|", i0)', n, n, n
  print '(f8.2, "|", es10.3, "|", e10.3)', d, d, d
  print '(g0, "|", g0, "|", g0)', n, d, m
  print '(a, "|", a6, "|", 12)', 'Sun', 'Sun', .true.
  print '(i3, 4x, i3, /, i3)', 1, 2, 3
end program descriptors
```

```
$ gfortran descriptors.f90 && ./a.out
   42| 0042|42
  456.79| 4.568E+02| 0.457E+03
42|456.789001|0.198900001E+31
Sun|   Sun| T
  1      2
  3
```

Field widths, repeats, and groups

- Width 0 is the minimum width: `i0`, `f0.2`. Good in text, but columns will not line up, and gfortran drops the zero before the decimal point.
- A value too wide for its field gives `***`.
- Repeat counts: `3f8.2`. Groups: `3(f5.2, ";")`.
- `*(...)` repeats a group as often as needed: one format for an array of any size.
- When the format runs out, the output goes on to a new line and reuses the last group.

```
program widths
  implicit none
  real :: r(5) = [0.5, -2.5, 3.25, 10.0, 12345.6]
  print '(i3, "|", i0, "|", f6.1)', 12345, 12345, r(5)
  print '(3f8.2)', r(1:3)
  print '(3(f5.2, ";"))', r(1:3)
  print '(a, *(1x, f0.2))', 'r =', r
  print '(2f8.2)', r
end program widths
```

```
$ gfortran widths.f90 && ./a.out
***|12345|******
    0.50   -2.50    3.25
 0.50;-2.50; 3.25;
r = .50 -2.50 3.25 10.00 12345.60
    0.50   -2.50
    3.25   10.00
12345.60
```

Internal write and non-advancing output

- An *internal write* writes into a string, the reverse of an internal read.
- Use it to build file names: `'(a, i4.4, a)'` gives `snap_0010.dat`.
- The rest of the string is filled with spaces, so use `trim`.
- `advance='no'` keeps the line open. It needs an explicit format.

```
$ gfortran internal_write.f90 && ./a.out
snap_0010.dat
snap_0020.dat
snap_0030.dat
Steps: 1 2 3 4 5 finished
```

```
program internal_write
  implicit none
  character(len=100) :: fname
  integer :: i
  do i = 1, 3
    write(fname, '(a, i4.4, a)') 'snap_', 10*i, '.dat'
    print '(a)', trim(fname)
  end do
  write(*, '(a)', advance='no') 'Steps:'
  do i = 1, 5
    write(*, '(1x, i0)', advance='no') i
  end do
  write(*, '(a)') ' finished'
end program internal_write
```

Files

Opening and closing files

- `open` connects a file to a unit, and `close` disconnects it.
- `newunit=u`: the compiler picks a free unit. Avoid fixed numbers like 10.
- `status='old'`, `'new'`, or `'replace'`. `action='read'`, `'write'`, or `'readwrite'`. `position='append'` writes at the end.
- Check `iostat`: missing files and wrong paths are the most common I/O errors.

```
program write_table
  implicit none
  real, parameter :: a(3) = [0.39, 0.72, 1.00]
  character(len=200) :: msg
  integer :: u, ios, i
  open(newunit=u, file='orbits.txt', status='replace', &
       action='write', iostat=ios, iomsg=msg)
  if (ios /= 0) error stop trim(msg)
  write(u, '(a)') '# a [AU]  P [yr]'
  do i = 1, size(a)
    write(u, '(f8.2, f8.3)') a(i), a(i)**1.5
  end do
  close(u)
end program write_table
```

```
$ gfortran write_table.f90 && ./a.out && cat orbits.txt
# a [AU]  P [yr]
    0.39   0.244
    0.72   0.611
    1.00   1.000
```

Reading a file of unknown length

- A file is read like the keyboard, with the unit in place of `*`.
- `iostat_end` (in `iso_fortran_env`) is the `iostat` value at the end of a file.
- `read(u, *)` with no variables skips a line, and `rewind(u)` goes back to the start.
- Two passes allocate the arrays once, instead of growing them for each value (Lecture 5).

```
$ gfortran read_table.f90 && ./a.out
3 planets, max P 1.00
```

```
program read_table
  use iso_fortran_env, only: iostat_end
  implicit none
  real, allocatable :: a(:), p(:)
  integer :: u, ios, n, i
  open(newunit=u, file='orbits.txt', action='read')
  n = -1                        ! do not count the header
  do                            ! 1st pass: count lines
    read(u, *, iostat=ios)
    if (ios == iostat_end) exit
    n = n + 1
  end do
  allocate(a(n), p(n))
  rewind(u)                     ! 2nd pass: read values
  read(u, *)                    ! skip the header
  do i = 1, n
    read(u, *) a(i), p(i)
  end do
  close(u)
  print '(i0, " planets, max P ", f0.2)', n, maxval(p)
end program read_table
```

Checking files and reporting errors

- `inquire(file=name, exist=found)` checks whether a file exists.
- `iso_fortran_env` names the standard units: `input_unit`, `output_unit`, and `error_unit`.
- Messages written to `error_unit` stay on the terminal when the output is redirected.
- `flush(u)` writes buffered output to the file now, so you can follow a long run.

```
program check_file
  use iso_fortran_env, only: error_unit
  implicit none
  logical :: found
  inquire(file='input.dat', exist=found)
  if (.not. found) then
    write(error_unit, '(a)') 'Error: input.dat not found'
    stop 1
  end if
  print '(a)', 'Found input.dat'
end program check_file
```

```
$ gfortran check_file.f90
$ ./a.out > out.txt
Error: input.dat not found
STOP 1
```

Command-line arguments

- `command_argument_count()` gives the number of arguments.
- `call get_command_argument(i, arg)` copies argument `i` into `arg` (0 is the program name).
- Arguments are text: convert numbers with an internal read.

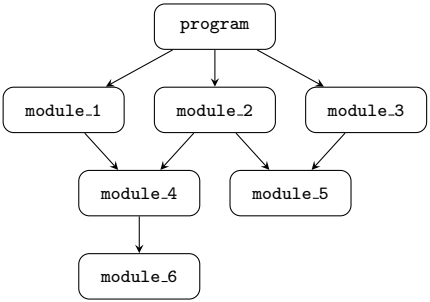
```
$ gfortran -o args args.f90
$ ./args
Usage: ./args <input file> <steps>
STOP 1
$ ./args input.dat 1000
Reading input.dat, running 1000 steps
```

```
program args
  implicit none
  character(len=256) :: fname, arg
  integer :: nsteps
  if (command_argument_count() /= 2) then
    print '(a)', 'Usage: ./args <input file> <steps>'
    stop 1
  end if
  call get_command_argument(1, fname)
  call get_command_argument(2, arg)
  read(arg, *) nsteps
  print '(3a, i0, a)', 'Reading ', trim(fname), &
                       ', running ', nsteps, ' steps'
end program args
```

Makefiles

Why build tools?

- A real program has many modules, and modules use other modules that use other modules...
- A module must be compiled before any code that uses it, because the compiler reads its `.mod` file.
- When a module changes, everything that uses it must be recompiled, and the program linked again.
- You do not want to do this by hand. A build tool does it for you, and recompiles only what has changed.



An arrow means "uses". If `module_6` changes, `module_4`, `module_1`, `module_2`, and the program must all be recompiled.

Make and makefile rules

- `make` runs the commands needed to build a target, using the rules in a file called `makefile`.
- A rule has a target, its dependencies, and the commands that make the target:

  ```
  target: dep_1 dep_2
      command
  ```

- Commands must start with a **TAB**, not spaces. Text copied from a PDF has spaces!
- A target is rebuilt only if it is missing or older than one of its dependencies.
- `make` builds the first target, and `make clean` the target `clean`.

```
all: e1

e1: m1.o e1.o
    gfortran -o e1 m1.o e1.o

m1.o: m1.f90
    gfortran -c m1.f90

# e1.f90 uses the module m1
e1.o: e1.f90 m1.o
    gfortran -c e1.f90

clean:
    rm -f *.o *.mod e1
```

Running make

- `make` prints each command before it runs it.
- A second `make` does nothing, because every target is newer than its dependencies.
- `touch` updates the modification time of a file, as if it had been edited. After `touch m1.f90`, `make` recompiles `m1.o`, then `e1.o`, which depends on it, and links the program again.
- The example makefiles are in `examples/ex08_makefiles`.

```
$ make
gfortran -c m1.f90
gfortran -c e1.f90
gfortran -o e1 m1.o e1.o
$ make
make: Nothing to be done for 'all'.
$ touch m1.f90
$ make
gfortran -c m1.f90
gfortran -c e1.f90
gfortran -o e1 m1.o e1.o
$ make clean
rm -f *.o *.mod e1
```

Variables and automatic variables

- `NAME = value` defines a variable, and `$(NAME)` is replaced by its value.
- By convention, `FC` is the Fortran compiler and `FCFLAGS` holds the compiler options.
- Variables can be overridden on the command line: `make FCFLAGS="-g -O0"`.
- Automatic variables, set for each rule:
  - `$@`: the target,
  - `$<`: the first dependency,
  - `$^`: all the dependencies.

```
FC = gfortran
FCFLAGS = -Wall -O2
OBJECTS = m1.o e1.o

e1: $(OBJECTS)
    $(FC) -o $@ $^

m1.o: m1.f90
    $(FC) $(FCFLAGS) -c $<

e1.o: e1.f90 m1.o
    $(FC) $(FCFLAGS) -c $<
```

```
$ make
gfortran -Wall -O2 -c m1.f90
gfortran -Wall -O2 -c e1.f90
gfortran -o e1 m1.o e1.o
```

Pattern rules and module dependencies

- `%.o: %.f90` is a pattern rule: it makes any `.o` file from the `.f90` file with the same name.
- It does not know which modules a file uses. Add a dependency line for each file that uses a module. Here `e1` uses `m1`, which uses `m2`.
- `make` then builds in the right order, whatever the order of `OBJECTS`. Without these lines:

```
gfortran -Wall -O2 -c e1.f90
Fatal Error: Cannot open module file 'm1.mod'
```

- `.PHONY` marks targets that are not files.

```
FC = gfortran
FCFLAGS = -Wall -O2
OBJECTS = e1.o m1.o m2.o

e1: $(OBJECTS)
    $(FC) -o $@ $^

%.o: %.f90
    $(FC) $(FCFLAGS) -c $<

# Module dependencies
e1.o: m1.o
m1.o: m2.o

.PHONY: clean test
clean:
    rm -f *.o *.mod e1
test: e1
    ./e1
```

More about build tools

Make

+ Available on Linux and macOS, and good for learning and for small projects.
– Several dialects that differ in details (GNU make is the most common one).
– Hard to maintain and slow in large projects, and not Windows-friendly.

Tutorials

- fortran-lang.org: Using make
- The GNU make manual

Modern build tools

- fpm, the Fortran Package Manager, finds the module dependencies by itself.
- CMake and Meson are used in large projects in many languages.
- Fortran Wiki: list of build tools

Exercise 1: formatted output

Tasks

1. Write a program that prints a table of the planets from Mercury to Neptune, with the name, the semi-major axis a in AU, and the orbital period $P = a^{3/2}$ in years and in days. Give the table a header line and make the columns line up. Use one named format constant for the rows.
2. Print the periods in seconds with `f`, `es`, and `g0`. Which one is the easiest to read?
3. Make one field too narrow. What happens?
4. Print all the periods on one line with one `print` statement and a `*(...)` format.

Exercise 2: files and arguments

Tasks

1. Change the program of Exercise 1 to write the table into a file whose name is given as a command-line argument. Print a usage message if the argument is missing.
2. Write a second program that takes the same file name as an argument, reads the table into allocatable arrays (skipping the header), and prints the mean period. What happens if the file does not exist? Report the error to `error_unit`.
3. For each planet, write the positions $x = a\cos(2\pi t/P)$, $y = a\sin(2\pi t/P)$ at 100 times over one orbit into its own file, `orbit_01.dat`, `orbit_02.dat`, ..., with the columns t, x, and y.

Exercise 3: makefiles

Tasks

1. Write a makefile for the `mymath` exercise of Lecture 4 (`mymath.f90` and `test_mymath.f90`), with explicit rules and no variables.
2. Run `make` twice. Then `touch test_mymath.f90` and run it again, and do the same with `mymath.f90`. Which files are recompiled each time, and why?
3. Rewrite the makefile with variables, automatic variables, and a pattern rule. Add the targets `clean` and `test`.
4. Move the `stats` module of Lecture 4 into its own file and make `test_mymath` use it too. Add the dependencies, and check that `make clean` followed by `make -j4` still works.